

Yaswanth Ravi Teja Dammalapati

Ashburn, VA | 📞 706-715-0830 | ✉ yaswanth.rtd2@gmail.com

🌐 [linkedin.com/yaswantravitejadammalapati](https://www.linkedin.com/in/yaswantravitejadammalapati) | 🐙 github.com/Yash8576 | leetcode.com/Yash_11844 | [Portfolio](#)

SUMMARY

Recent **MS Computer Science graduate** from the University of Georgia with end-to-end ownership across the full SDLC - requirements, architecture, hardware/software design, backend APIs, cloud deployment, observability, production rollout, and iteration from stakeholder feedback. Built production **IoT infrastructure** for UGA research facilities using **ESP32, Raspberry Pi, Docker, InfluxDB, Grafana, and Firebase**, and shipped full-stack products across social commerce, coding-problem guidance, real-time systems, cloud APIs, and **AI/RAG workflows**. Seeking **Software Engineer** roles to apply strong system design, backend, cloud, and product engineering skills to build reliable, user-facing platforms.

EDUCATION

University of Georgia

Aug 2024 – May 2026

Master of Science in Computer Science

GPA: 3.82 / 4.0

National Institute of Technology, Raipur

Dec 2020 – Jun 2024

Bachelor of Technology in Information Technology

CGPA: 8.21 / 10.0

EXPERIENCE

Graduate Research Assistant - IoT & Environmental Monitoring Systems

Aug 2025 – May 2026

University of Georgia

Athens, GA

- Served as the **sole CS engineer** on a cross-disciplinary environmental monitoring team, leading a full technical rebuild from scratch, replacing a fragile **Python/Flask** laptop workflow and raw CSV files with a production-grade edge-computing stack.
- Redesigned the **ESP32**-based hardware end-to-end: reselected sensors/components, overhauled PCB layout and enclosure geometry, **reduced device footprint by 50%** enabling easier deployment across space-constrained facility environments.
- Improved **sensor measurement accuracy by 35%** through targeted calibration routines, optimized sensor placement, and repeated field validation across greenhouses, growth chambers, and poultry farms, eliminating systematic drift that skewed prior research data.
- Architected and deployed a production **Raspberry Pi** edge cluster containerized with **Docker**, integrating **InfluxDB** for time-series storage, **Grafana** for live dashboards, and **Firebase** for cloud-backed data handoff; **improved data logging reliability** by replacing ad-hoc scripts with a robust ingestion and persistence pipeline.
- Implemented a hybrid mesh communication pipeline, **ESP-NOW** for nearby device streaming to edge gateways, Wi-Fi for Firebase-enabled devices, and Raspberry Pi loggers for ingestion, normalization, and persistence, **reducing data loss events by over 80%** compared to the prior single-path Wi-Fi setup.
- Documented end-to-end operational workflows for non-engineering staff, enabling the system to run without CS oversight after handoff.

PROJECTS

BuzzCart | *Flutter, Dart, Go/Gin, PostgreSQL, Firebase/GCS, Redis, FastAPI, Docker, GCP*

[Live Link](#)

- Drove BuzzCart from idea to **live MVP**: a trust-centered social commerce platform that surfaces whether a user's connections bought/reviewed a product before purchase, directly countering anonymous-review uncertainty.
- Built the complete **Flutter** product surface: auth, social feed, reel, videos, seller product publishing, media upload, shop/detail pages, cart/checkout, buyer visibility, profiles, and settings.
- Engineered a **Go** backend with domain handlers for auth, products, reviews, cart, checkout, feed, uploads, follow graph, messages, and media; secured with **JWT**, middleware validation, role-based checks, and structured API contracts.
- Implemented trust logic at the data layer: order-linked verified-review eligibility, duplicate-review prevention, helpful votes, moderation states, privacy-aware buyer lists, and social ranking by follow graph, recency, and usefulness.
- Designed production architecture across **Cloudflare Pages, GCP Cloud Run, Cloud SQL PostgreSQL, Firebase/GCS** storage, optional **Redis** caching, Docker Compose, and **GitHub Actions** CI/CD.
- Built a **FastAPI** RAG service using **FAISS**, Sentence Transformers, token-aware chunking, dense+lexical hybrid retrieval, neighbor expansion, cross-encoder reranking, and grounded answers with source metadata.
- Validated product-market and deployment decisions through direct conversations with **CEOs and founders from big-tech**, translating feedback into deployment, trust, and onboarding priorities.
- Designed real-time **WebSocket**-based messaging and notification infrastructure for buyer-seller interactions, follow-event fanout, and cart/checkout status updates, with graceful polling fallback for low-connectivity clients.
- Established an end-to-end **media pipeline**: client-side compression, signed **GCS upload URLs**, post-upload processing triggers, CDN-backed delivery, and lifecycle policies for storage cost control across video, image, and document assets.

- Built NexAlgo as a coding-problem guidance platform plus **Chrome Manifest V3** extension that auto-detects LeetCode/GeeksforGeeks pages and surfaces contextual hints, approaches, and solutions without leaving the workspace.
- Developed the **Next.js/React** app with **Firebase Auth**, session bootstrap, **RBAC** (admin/editor/viewer), multi-filter problem browsing, per-user progress tracking, and rich problem pages: hints, intuition, code walkthroughs, complexity analysis, and company tags.
- Modeled the data layer in **PostgreSQL** via **Prisma**: users, roles, problems, sources, submissions, generated content, reviews, preferences, and progress, enabling full auditability across published and draft content.
- Engineered **TypeScript Express REST APIs** for listing/lookup, source-key normalization, authenticated submissions, review queues, approve/reject workflows, and regeneration, with Firebase bearer-token auth and RBAC enforced throughout.
- Built Chrome extension content scripts to scrape normalized problem metadata (statement, difficulty, topics, companies, URL); background workers call NexAlgo APIs, refresh sessions, submit missing problems, and sync side-panel state.
- Integrated **OpenAI Responses API** with strict **JSON-schema** output to generate stepwise hints, intuition, approach, walkthroughs, time/space complexity, and Python/Java/C++ solutions, gated behind an editor approval workflow before user-facing publishing.
- Designed the production deployment path with a dedicated **Cloud Run** backend, **Cloud SQL PostgreSQL**, Firebase Admin token verification, and extension packaging scripts, keeping generated content fully controlled before reaching users.
- Implemented a **review queue workflow** where AI-generated content enters a draft state, editors can approve, reject, or trigger regeneration, and only approved content is surfaced to users, ensuring quality and reducing hallucination risk at scale.

TECHNICAL SKILLS

Languages: Go, TypeScript, JavaScript, Dart, Python, Java, C++, SQL, Shell/Bash

Frontend & Mobile: Flutter, Next.js, React, Chrome Extensions (Manifest V3), Responsive Web/Mobile UI, State Management

Backend & APIs: Go/Gin, Node.js/Express, FastAPI, REST APIs, WebSockets, JWT, RBAC, Firebase Auth, OpenAI Responses API

Data & AI Systems: PostgreSQL, Cloud SQL, Prisma, Redis, Firebase/GCS, MinIO, RAG, FAISS, Vector Search, Embeddings, Hybrid Retrieval, Reranking

Cloud & DevOps: GCP Cloud Run, Cloudflare Pages, Docker, Kubernetes, GitHub Actions, CI/CD, Linux

IoT & Embedded: ESP32, Raspberry Pi, ESP-NOW Mesh, InfluxDB, Grafana, Sensor Calibration, Edge Computing

System Design: Microservices, Distributed Systems, Caching, Object Storage, Real-Time Messaging, Auth/Authorization, API Contracts, Scalability, Observability